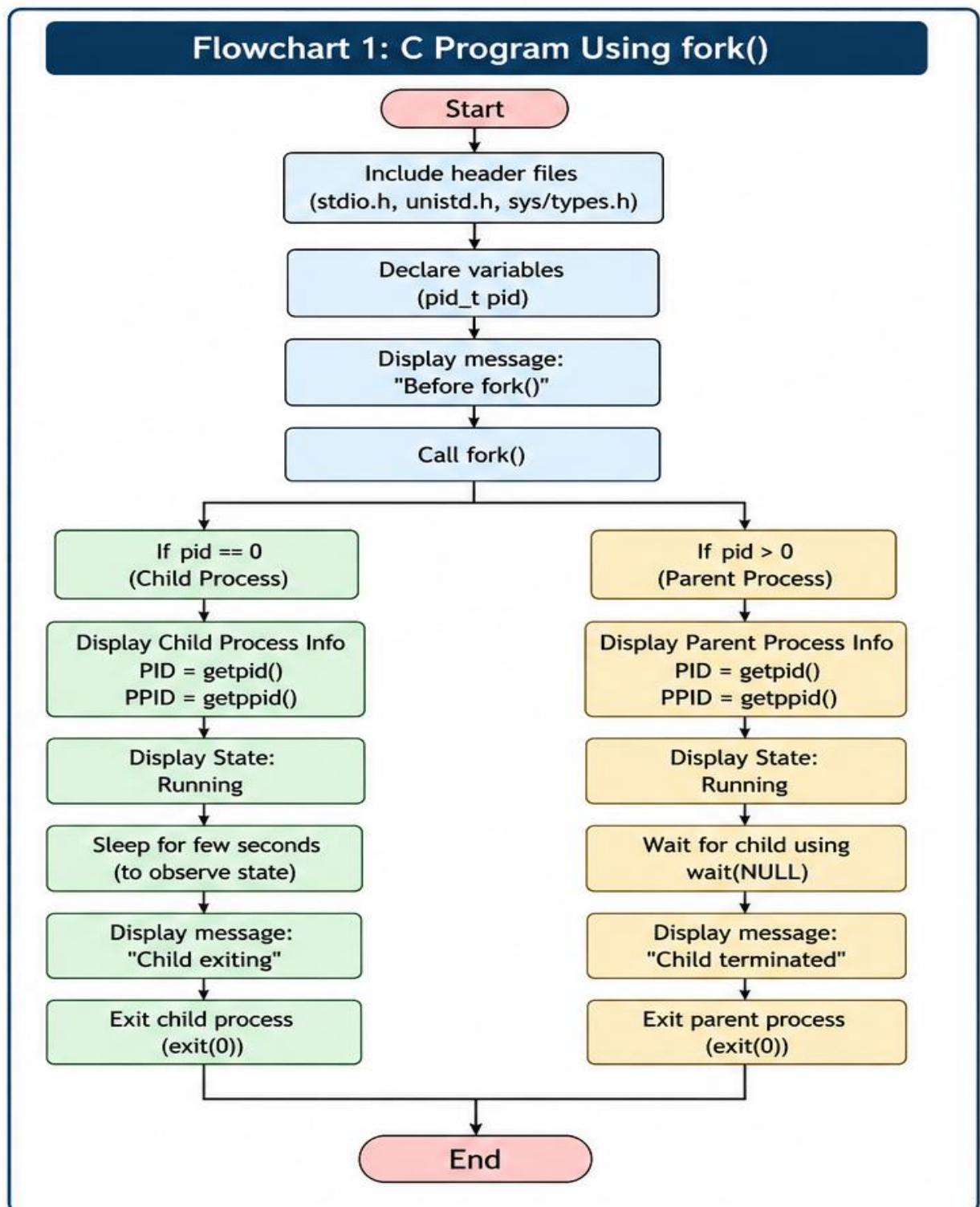


Task 3: Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Flow Chart



```

divya@Divyasree:~$ cd ossp
-bash: cd: ossp: No such file or directory
divya@Divyasree:~$ cd OSSP
divya@Divyasree:~/OSSP$ pwd
/home/divya/OSSP
divya@Divyasree:~/OSSP$ mkdir Practical3
divya@Divyasree:~/OSSP$ cd Practical3
divya@Divyasree:~/OSSP/Practical3$ mkdir Task3
divya@Divyasree:~/OSSP/Practical3$ cd Task3
divya@Divyasree:~/OSSP/Practical3/Task3$ nano task3.c
divya@Divyasree:~/OSSP/Practical3/Task3$ gcc task3.c -o task3
divya@Divyasree:~/OSSP/Practical3/Task3$ ./task3
Parent process started.
Parent PID: 553
Parent PPID: 390

--- Parent Process ---
Parent PID: 553
Parent PPID: 390
Parent State: Running
Parent is waiting for child to finish...

--- Child Process ---
Child PID: 554
Child PPID: 553
Child State: Running
Child State: Terminating
Child has terminated.
Parent State: Running
Parent State: Terminating
divya@Divyasree:~/OSSP/Practical3/Task3$ ps -f
UID          PID     PPID  C  STIME TTY          TIME CMD
divya         390       386  0  17:45 pts/0        00:00:00 -bash

Parent State: Terminating
divya@Divyasree:~/OSSP/Practical3/Task3$ ps -f
UID          PID     PPID  C  STIME TTY          TIME CMD
divya         390       386  0  17:45 pts/0        00:00:00 -bash
divya         557       390  0  17:53 pts/0        00:00:00 ps -f
divya@Divyasree:~/OSSP/Practical3/Task3$ ps -ef | grep task3
divya         561       390  0  17:53 pts/0        00:00:00 grep --color=auto task3
divya@Divyasree:~/OSSP/Practical3/Task3$ |

```

Source code :

```
GNU nano 7.2 task3.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Parent process started.\n");
    printf("Parent PID: %d\n", getpid());
    printf("Parent PPID: %d\n", getppid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("\n--- Child Process ---\n");
        printf("Child PID: %d\n", getpid());
        printf("Child PPID: %d\n", getppid());
        printf("Child State: Running\n");

        sleep(5);
```

```
        printf("Child State: Terminating\n");
        exit(0);
    }
    else {
        // Parent process
        printf("\n--- Parent Process ---\n");
        printf("Parent PID: %d\n", getpid());
        printf("Parent PPID: %d\n", getppid());
        printf("Parent State: Running\n");

        printf("Parent is waiting for child to finish...\n");

        wait(NULL);

        printf("Child has terminated.\n");
        printf("Parent State: Running\n");
        printf("Parent State: Terminating\n");
    }

    return 0;
}
```